

QUESTIONS 19 / 20 completed

Shape-2 ▾

SHAPE-2

Write a Java program to implement a basic shape hierarchy using polymorphism. Create a parent class called Shape with attributes such as area and perimeter. Then, create child classes such as Circle and Rectangle that inherit from the Shape class and have their own unique methods for calculating area and perimeter.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     static class Shape {
3         double area(){ return 0; }
4         double perimeter(){ return 0; }
5     }
6     static class Circle extends Shape {
7         double r;
8         Circle(double r){ this.r = r; }
9         double area(){ return Math.PI * r * r; }
10        double perimeter(){ return 2 * Math.PI * r; }
11    }
12    static class Rectangle extends Shape {
13        double l, w;
14        Rectangle(double l, double w){ this.l = l; this.w = w; }
15        double area(){ return l * w; }
16        double perimeter(){ return 2 * (l + w); }
17    }
18    public static void main(String[] args) {
19        Shape[] s = { new Circle(5), new Rectangle(4, 3) };
20        for (Shape x : s) System.out.println("Area=" + x.area() + ", Perimeter=" + x.perimeter());
21    }
```

OUTPUT

```
Area=78.53981633974483, Perimeter=31.41592653589793
Area=12.0, Perimeter=14.0
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Draw ▾

DRAW

Write a Java program to demonstrate polymorphism with interfaces. Create an interface called Drawable with a method called draw(). Then, create classes such as Circle and Square that implement the Drawable interface and have their own unique implementations of the draw() method.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Drawable shape1 = new Circle();
4         Drawable shape2 = new Square();
5         shape1.draw();
6         shape2.draw();
7         Drawable[] shapes = { new Circle(), new Square() };
8         for (Drawable shape : shapes) {
9             shape.draw();
10        }
11    }
12 }
13 interface Drawable {
14     void draw();
15 }
16 class Circle implements Drawable {
17     public void draw() {
18         System.out.println("Drawing a Circle: O");
19     }
20 }
21 class Square implements Drawable {
```

OUTPUT

```
Drawing a Circle: O
Drawing a Square: []
Drawing a Circle: O
Drawing a Square: []
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Animal-2

ANIMAL-2

Write a Java program to demonstrate polymorphism with abstract classes. Create an abstract class called Animal with abstract methods such as eat() and sleep(). Then, create child classes such as Dog and Cat that inherit from the Animal class and implement their own versions of the eat() and sleep() methods.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Animal myDog = new Dog();
4         Animal myCat = new Cat();
5         System.out.println("Dog behavior:");
6         myDog.eat();
7         myDog.sleep();
8         System.out.println("\nCat behavior:");
9         myCat.eat();
10        myCat.sleep();
11    }
12 }
13 abstract class Animal {
14     abstract void eat();
15     abstract void sleep();
16 }
17 class Dog extends Animal {
18     void eat() {
19         System.out.println("Dog eats bones and meat.");
20     }
21     void sleep() {
```

OUTPUT

```
System Error: Could not reach compiler or server error occurred.
```

INPUT

Your INPUT goes here!

QUESTIONS 19 / 20 completed

Overloading ▾

OVERLOADING

Write a Java program to demonstrate method overloading with variable arguments. Create a method called sum() that takes in a variable number of integers and returns their sum. Then, create an overloaded version of the sum() method that takes in a variable number of doubles and returns their sum.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class OverloadingDemo {
2     public static int sum(int... numbers) {
3         int total = 0;
4         for (int num : numbers) {
5             total += num;
6         }
7         return total;
8     }
9     public static double sum(double... numbers) {
10        double total = 0.0;
11        for (double num : numbers) {
12            total += num;
13        }
14        return total;
15    }
16    public static void main(String[] args) {
17        int intSum = sum(10, 20, 30, 40);
18        System.out.println("Sum of integers: " + intSum);
19        double doubleSum = sum(10.5, 20.2, 30.3);
20        System.out.println("Sum of doubles: " + doubleSum);
21    }
```

OUTPUT

```
Sum of integers: 100
Sum of doubles: 61.0
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Overriding ▾

OVERRIDING

Write a Java program to demonstrate method overriding with a simple calculator. Create a parent class called Calculator with methods such as add(), subtract(), multiply(), and divide(). Then, create a child class called ScientificCalculator that overrides the multiply() method to perform a more complex calculation.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Calculator calc = new Calculator();
4         ScientificCalculator sciCalc = new ScientificCalculator();
5         double x = 10;
6         double y = 5;
7         System.out.println("Calculator:");
8         System.out.println("Add: " + calc.add(x, y));
9         System.out.println("Subtract: " + calc.subtract(x, y));
10        System.out.println("Multiply: " + calc.multiply(x, y));
11        System.out.println("Divide: " + calc.divide(x, y));
12        System.out.println("\nScientificCalculator (Overriding multiply):");
13        System.out.println("Add: " + sciCalc.add(x, y));
14        System.out.println("Subtract: " + sciCalc.subtract(x, y));
15        System.out.println("Multiply: " + sciCalc.multiply(x, y));
16        System.out.println("Divide: " + sciCalc.divide(x, y));
17    }
18 }
19 class Calculator {
20     public double add(double a, double b) {
21         return a + b;
```

OUTPUT

```
Calculator:
Add: 15.0
Subtract: 5.0
Multiply: 50.0
Divide: 2.0

ScientificCalculator (Overriding multiply):
Add: 15.0
Subtract: 5.0
Multiply: 65.0
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Person ▾

PERSON

Write a Java program to create a class called Person with a constructor that takes in a name and age, and a method to print out the person's name and age.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Person {
2     private String name;
3     private int age;
4     public Person(String name, int age) {
5         this.name = name;
6         this.age = age;
7     }
8     public void printDetails() {
9         System.out.println("Name: " + name);
10        System.out.println("Age: " + age);
11    }
12    public static void main(String[] args) {
13        Person person = new Person("Moshika", 19);
14        person.printDetails();
15    }
16 }
```

OUTPUT

```
Name: Moshika
Age: 20
```

INPUT

Your INPUT goes here!

QUESTIONS 19 / 20 completed

Rect-Const ▾

RECT-CONST

Write a Java program to create a class called Rectangle with a constructor that takes in the length and width of the rectangle, and a method to calculate and return the area of the rectangle.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {  
2     private double length;  
3     private double width;  
4     public Main(double length, double width) {  
5         this.length = length;  
6         this.width = width;  
7     }  
8     public double area() {  
9         return length * width;  
10    }  
11    public static void main(String[] args) {  
12        Main rect = new Main(10, 5);  
13        System.out.println("Area of rectangle: " + rect.area());  
14    }  
15 }
```

OUTPUT

```
Area of rectangle: 50.0
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Bank-Const ▾

BANK-CONST

Write a Java program to create a class called BankAccount with a constructor that takes in an account number and an initial balance, and methods to deposit and withdraw money from the account.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         BankAccount myAccount = new BankAccount("SAV12345", 1000.0);
5         myAccount.displayBalance();
6         myAccount.deposit(500.0);
7         myAccount.withdraw(200.0);
8         myAccount.displayBalance();
9         myAccount.withdraw(2000.0);
10    }
11 }
12 class BankAccount {
13     private String accountNumber;
14     private double balance;
15     public BankAccount(String accountNumber, double initialBalance) {
16         this.accountNumber = accountNumber;
17         this.balance = initialBalance;
18     }
19     public void deposit(double amount) {
20         if (amount > 0) {
21             balance += amount;
```

OUTPUT

```
Account Number: SAV12345
Current Balance: 1000.0
Successfully deposited: 500.0
Successfully withdrawn: 200.0
Account Number: SAV12345
Current Balance: 1300.0
Insufficient balance!
```

INPUT

Your INPUT goes
here!

QUESTIONS 19 / 20 completed

Car-Const ▾

CAR-CONST

Write a Java program to create a class called Car with a constructor that takes in the make, model, and year of the car, and a method to print out the car's make, model, and year.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 public class Main {
2     static class Car {
3         private String make;
4         private String model;
5         private int year;
6         public Car(String make, String model, int year) {
7             this.make = make;
8             this.model = model;
9             this.year = year;
10        }
11        public void printCarDetails() {
12            System.out.println("Make : " + make);
13            System.out.println("Model : " + model);
14            System.out.println("Year : " + year);
15        }
16    }
17    public static void main(String[] args) {
18        Car car = new Car("Toyota", "Corolla", 2020);
19        car.printCarDetails();
20    }
21 }
```

OUTPUT

```
Make: Toyota
Model: Corolla
Year: 2020
```

INPUT

Your INPUT goes here!

QUESTIONS 19 / 20 completed

Student-Const ▾

STUDENT-CONST

Write a Java program to create a class called Student with a constructor that takes in a name, ID number, and a list of grades, and methods to calculate and return the student's average grade and letter grade.

PROGRAM EDITOR

Java ▾

Run

Save

```
1 import java.util.List;
2 import java.util.ArrayList;
3 public class Student {
4     private String name;
5     private String idNumber;
6     private List<Double> grades;
7     public Student(String name, String idNumber, List<Double> grades) {
8         this.name = name;
9         this.idNumber = idNumber;
10        this.grades = grades;
11    }
12    public double calculateAverage() {
13        if (grades == null || grades.isEmpty()) {
14            return 0.0;
15        }
16        double sum = 0;
17        for (double grade : grades) {
18            sum += grade;
19        }
20        return sum / grades.size();
21    }
```

OUTPUT

```
Average Grade: 85.0
Letter Grade: B
```

INPUT

Your INPUT goes
here!